Software Design Document (SDD)

Module	UART Driver
Version	1.0.0
Team	Carlos Cayetano · Adrian Amaro · Adalberto Aguirre · Daniel López
Date	14/05/2026
Institution	CETYS Universidad

1. Introduction

1.1. Purpose

This document describes the design and architecture of the UART driver module for the STM32F411RE. It is based on the requirements in the SRS and covers the register-level configuration of USART2, the baud rate calculation, and the polling transmit loop. The Serial module that wraps this driver is also documented here because it is the primary interface used by the application in Lab Assignment 04.

1.2 Scope

The UART driver configures USART2 on the STM32F411RE for asynchronous, TX-only serial communication at 115200 baud. It relies on the 16 MHz HSI clock (default after reset) and PA2 as the transmit pin in Alternate Function 7. The Serial module wraps the driver and adds a printf-style formatting interface, used by main.c to send ADC sensor telemetry to a PC terminal over the ST-Link virtual COM port every 500 ms.

1.3 Definitions, Acronyms

Term	Definition
AF7	Alternate Function 7 — the GPIO alternate function number that routes PA2 to USART2_TX.
APB1	Advanced Peripheral Bus 1 — the bus that clocks USART2 on STM32F4 devices.
BRR	Baud Rate Register — holds the integer and fractional divider for the USART clock to produce the target baud rate.

CMSIS	Cortex Microcontroller Software Interface Standard — provides the USART_TypeDef and RCC_TypeDef register structs.
CR1	USART Control Register 1 — contains TE (transmitter enable) and UE (USART enable) bits.
DR	USART Data Register — the transmit holding register; writing a byte here starts transmission.
FR	Functional Requirement — traceability label from the SRS.
HSI	High-Speed Internal oscillator — 16 MHz default clock on the STM32F411RE after reset.
PA2	GPIO pin A2 on the STM32F411RE — connected to USART2_TX via Alternate Function 7.
SR	USART Status Register — contains TXE (transmit data register empty) and TC (transmission complete) flags.
SRS	Software Requirements Specification — the document this SDD traces to.
TE	Transmitter Enable bit (CR1 bit 3) — activates the USART transmitter path.
TX	Transmit — the data direction from the MCU to an external receiver.
TXE	Transmit Data Register Empty flag (SR bit 7) — set when DR is ready to accept a new byte.
UE	USART Enable bit (CR1 bit 13) — enables the USART peripheral.
UART	Universal Asynchronous Receiver-Transmitter — a serial communication protocol using start, data, and stop bits with no shared clock.
USART	Universal Synchronous/Asynchronous Receiver-Transmitter — the STM32 peripheral; used in asynchronous mode here.
USART2	The USART2 peripheral instance on the STM32F411RE, mapped to PA2 (TX) and PA3 (RX) and routed to the ST-Link VCP.

VCP	Virtual COM Port — the USB serial interface provided by the ST-Link debugger, appears as a COM port on the host PC.

1.4 Requirements

The functional requirements this module must satisfy are specified in SRS-UART_Driver. The table below maps each FR to the driver operation that satisfies it.

Requirement ID	Description
FR-1	The driver shall enable the USART2 peripheral clock via RCC_APB1ENR. Implemented in uart_init().
FR-2	The driver shall set the baud rate to 115200 baud using the formula BRR = fCK / baud_rate with a 16 MHz clock. Implemented in uart_init().
FR-3	The driver shall enable the USART transmitter (TE) and the peripheral (UE) by writing CR1. Implemented in uart_init().
FR-4	The driver shall wait until the TXE flag in SR is set before writing to DR. Implemented in uart_write().
FR-5	The driver shall write the byte to the USART2 data register (DR) to initiate transmission. Implemented in uart_write().

2. System Architecture

2.1 High-Level Design

Lab Assignment 04 adds a serial telemetry path on top of the sensor and timer infrastructure from earlier labs. The software stack for the UART subsystem has three levels:

- Application Layer: main.c initializes all modules and calls serial_printf() inside the timer loop to send ADC readings to the PC terminal at 500 ms intervals.
- Serial Layer (serial.c): wraps uart_init() and uart_write() and adds a variadic printf-style formatting function. It uses a 128-byte internal buffer and the utils module for integer-to-ASCII conversion.
- UART Driver Layer (uart.c): owns USART2 register configuration (PA2 alternate function, BRR, CR1) and provides a two-function API (uart_init, uart_write).
- Hardware Layer: STM32F411RE USART2 peripheral registers and PA2 GPIO, as defined in stm32f4xx.h and stm32f411xe.h.

2.2 Module Interfaces

The UART driver is declared in uart.h and depends on gpio_driver.h for gpio_initPort() and gpio_setAlternateFunction(). The driver calls gpio_initPort(GPIOA) to enable the GPIOA clock and gpio_setAlternateFunction(GPIOA, 2, 7) to route PA2 to AF7 (USART2_TX).

The Serial module is declared in serial.h. It includes uart.h for uart_init()/uart_write() and utils.h for integer conversion. serial_printf() uses a static 128-byte buffer (serial_buffer) and implements its own variadic format loop with va_list directly, since utils_snprintf() does not expose a va_list variant.

Key CMSIS register references: USART2->CR1 (TE, UE bits), USART2->BRR (baud rate divider), USART2->SR (TXE flag), USART2->DR (transmit data register), RCC->APB1ENR (USART2EN bit), GPIOA->MODER and GPIOA->AFR[0] (PA2 alternate function via gpio driver).

3. Detailed Design

3.1 Class Diagram

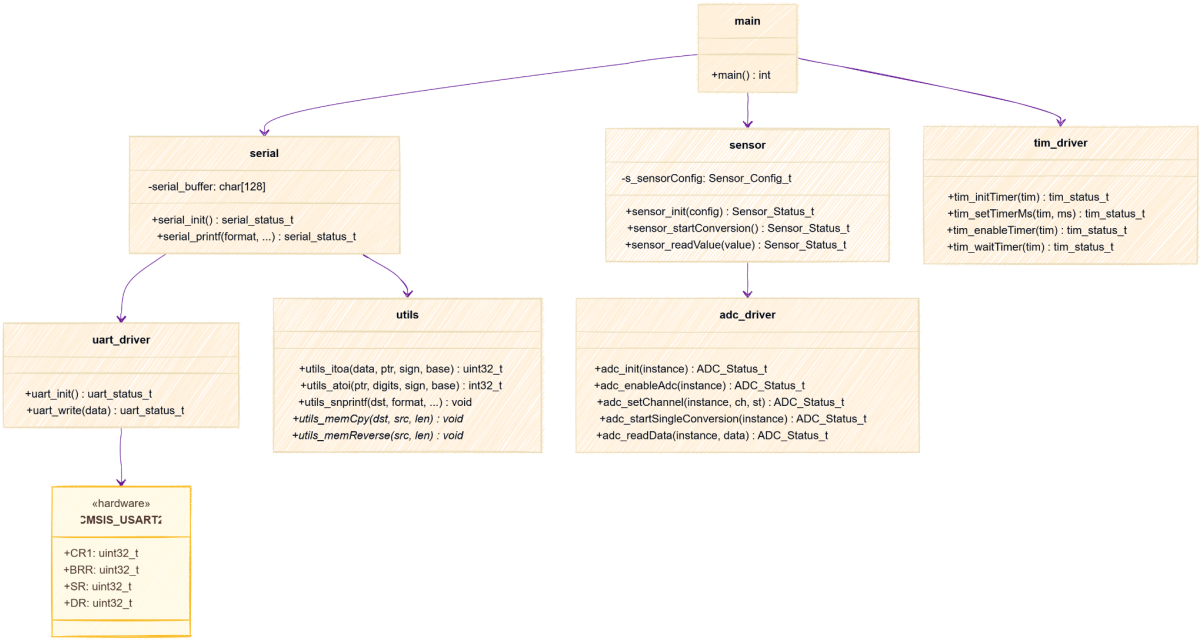

Figure 1. Class Diagram — UART Driver

3.2 Module Behavior

The eight figures below cover the UART initialization sequence, the transmit polling loop, the serial_printf formatting path, and the driver state machine.

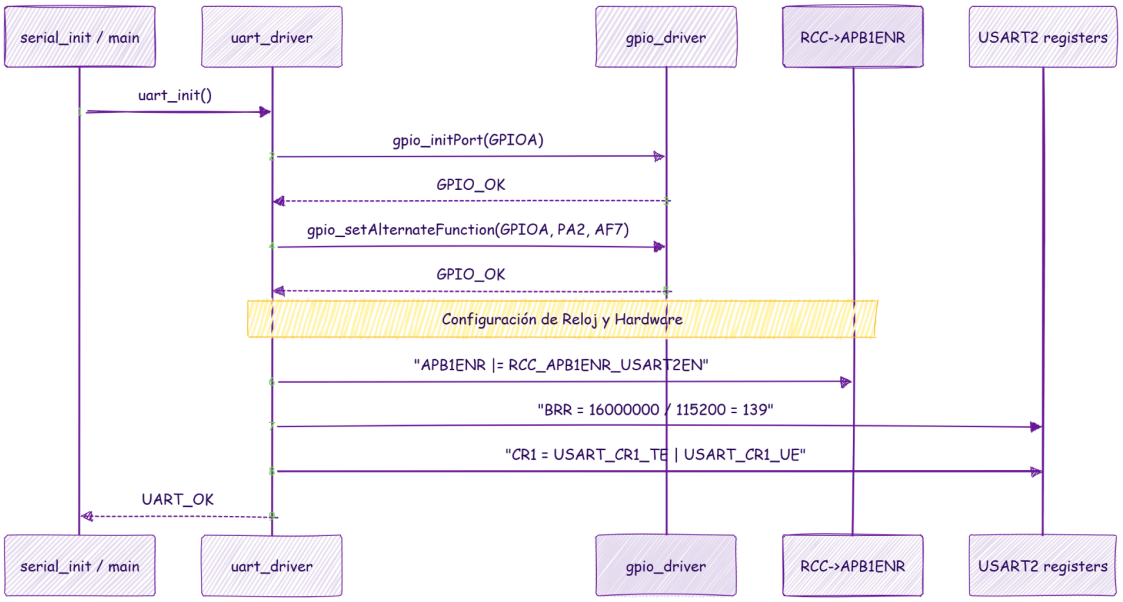

Figure 2. Sequence Diagram — uart_init

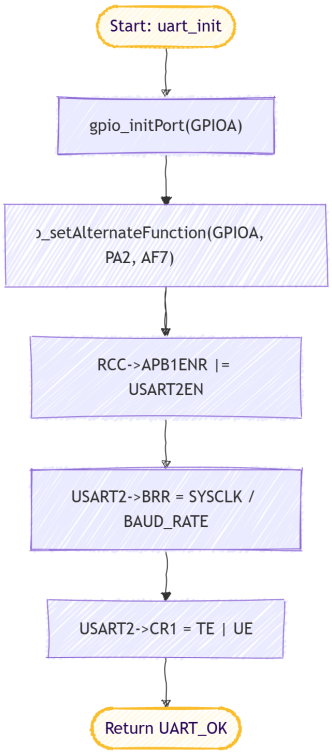

Figure 3. Activity Diagram — uart_init

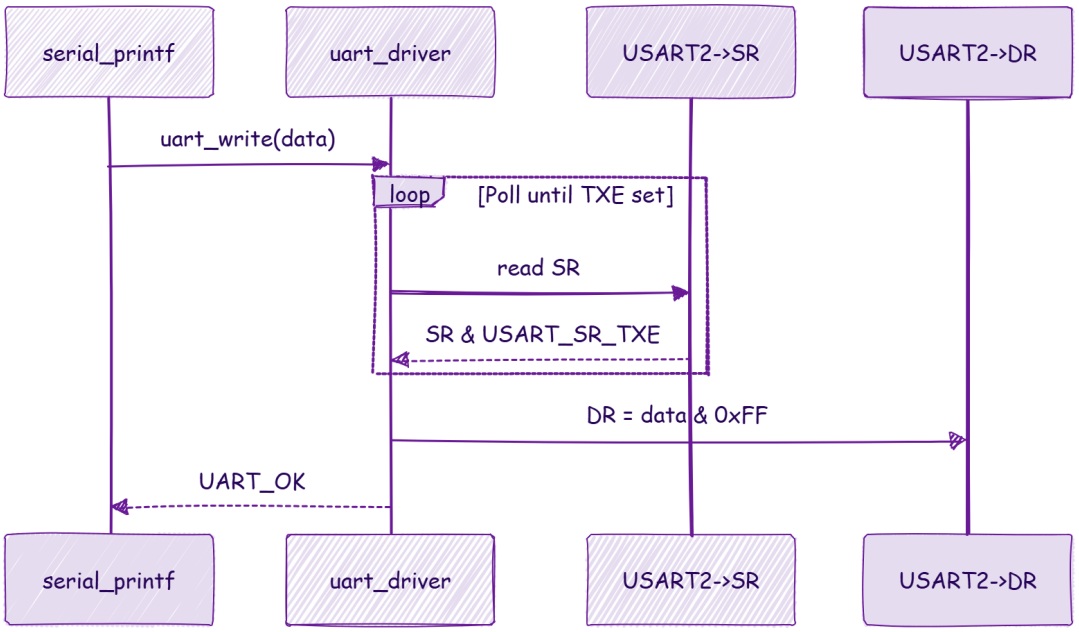

Figure 4. Sequence Diagram — uart_write

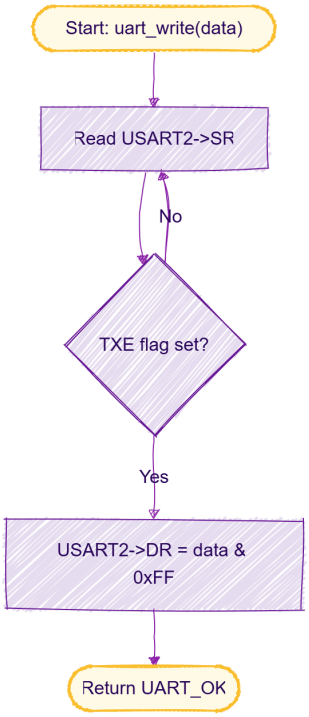

Figure 5. Activity Diagram — uart_write

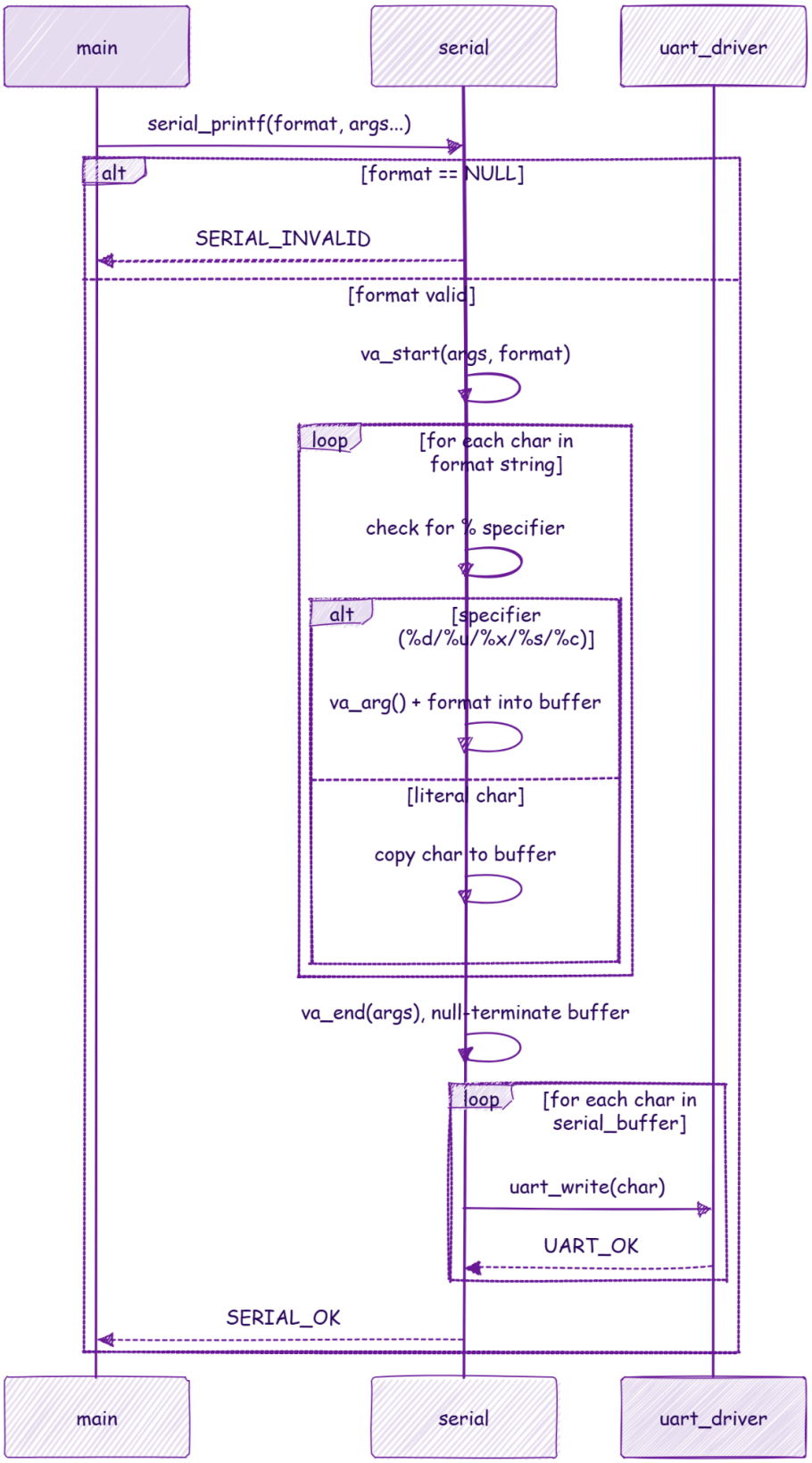

Figure 6. Sequence Diagram — serial_printf

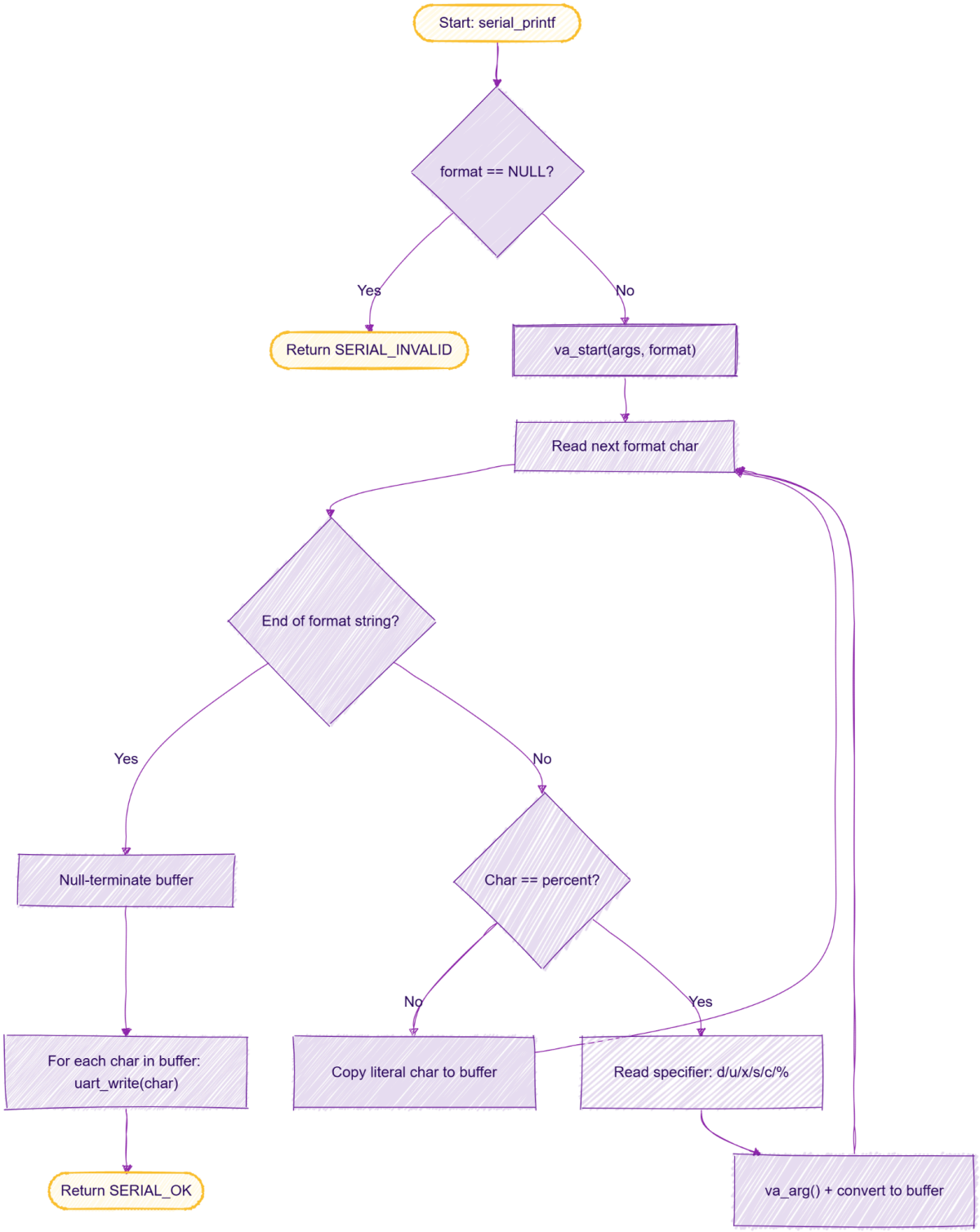

Figure 7. Activity Diagram — serial_printf

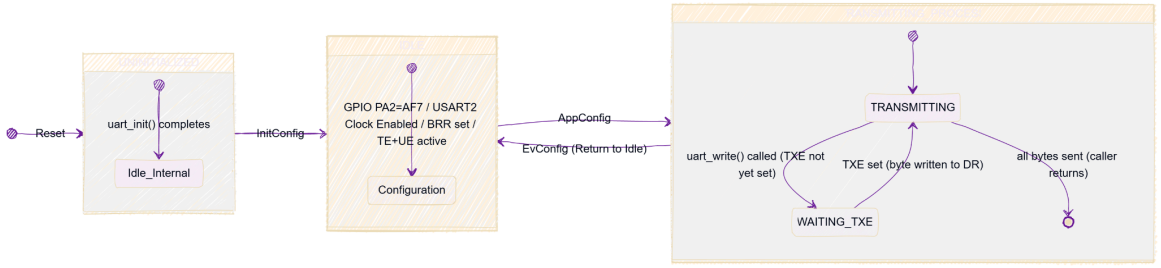

Figure 8. State Diagram — UART Driver

3.3 UART Driver

The UART driver is a two-function module: uart_init() configures the hardware once at startup, and uart_write() sends one byte at a time. There are no private helper functions or lookup tables; the implementation is intentionally minimal to match the TX-only requirement of Lab 04.

During initialization, the driver first calls gpio_initPort() to enable the GPIOA AHB1 clock, then gpio_setAlternateFunction() to set PA2 MODER to 0b10 (alternate function) and write AF7 into GPIOA->AFR[0] bits [11:8]. After the GPIO is configured, the driver enables the USART2 APB1 clock by setting RCC->APB1ENR bit 17. It then writes the baud rate divider: BRR = 16000000 / 115200, which gives 138.88, truncated by integer division to 139 (0x008B). Finally, it sets CR1 to USART_CR1_TE | USART_CR1_UE in

The uart_write() function is a blocking call. It polls SR until TXE (bit 7) is set, which confirms the hardware shift register is ready for a new byte, then writes the byte to DR. The hardware clears TXE automatically when DR is loaded. Using TXE rather than TC (Transmission Complete) allows back-to-back byte writes without waiting for the last stop bit to finish transmitting, which gives better throughput when sending strings.

The Serial module builds on uart_write() to provide a formatted print interface. serial_printf() opens a va_list, iterates through the format string, and dispatches each format specifier to utils_itoa() for numeric conversions or copies the character directly for %c and literal text. The result accumulates in a 128-byte static buffer, then each byte is sent with uart_write(). The 128-byte limit applies per call; strings longer than 127 characters are silently truncated.

In Lab Assignment 04, the main loop waits on the TIM2 hardware (tim_waitTimer) for 500 ms, reads the ADC potentiometer via sensor_startConversion() and sensor_readValue(), converts the raw 12-bit result to millivolts with the formula voltage_mV = (adc_raw * 3300) / 4095, and prints both values with serial_printf("ADC Value: %d | Voltage: %d mV\n", ...). The ST-Link VCP forwards the output to a PC terminal monitor at 115200 baud, 8N1.

4. Application Programming Interfaces

4.1 Data Types and Macros

uart_status_t: return type for uart_init() and uart_write(). Three values: UART_OK (0) — operation succeeded; UART_ERROR (1) — generic failure; UART_INVALID (2) — invalid argument.

serial_status_t: return type for serial_init() and serial_printf(). Same three values: SERIAL_OK (0), SERIAL_ERROR (1), SERIAL_INVALID (2).

Macros defined in uart.h: UART_SYSTEM_CLOCK_HZ = 16000000UL (HSI frequency); UART_BAUD_RATE = 115200UL. Private constants in uart.c: UART_TX_PORT = GPIOA, UART_TX_PIN = 2U, UART_TX_AF = 7U.

Macro defined in serial.h: SERIAL_BUFFER_SIZE = 128U — maximum formatted string length per serial_printf() call.

4.2 Error Codes

Code	Definition
UART_OK (0)	The operation completed successfully.
UART_ERROR (1)	A generic failure. Not currently used in this implementation since uart_init() and uart_write() always return UART_OK.
UART_INVALID (2)	An argument was out of range. Not currently used but reserved for future parameter validation.
SERIAL_OK (0)	Formatting and transmission completed without errors.
SERIAL_ERROR (1)	An internal error occurred in the serial layer.
SERIAL_INVALID (2)	The format string passed to serial_printf() was NULL.

4.3 Function Definitions

uart_init

Function Name	uart_init
Syntax	uart_status_t uart_init(void)
Parameters (in)	None

Parameters (inout)	None
Parameters (out)	None
Return value	uart_status_t — always returns UART_OK in the current implementation.
Description	Configures PA2 as USART2_TX (AF7), enables the USART2 APB1 clock, sets BRR to 139 (16 MHz / 115200), and writes CR1 = TE | UE.
Constraints	Assumes a 16 MHz HSI clock. If the system clock is changed, UART_SYSTEM_CLOCK_HZ must be updated to maintain the correct baud rate. Does not configure RX.
Available via	uart.h

uart_write

Function Name	uart_write
Syntax	uart_status_t uart_write(uint8_t data)
Parameters (in)	data — uint8_t: the byte to transmit.
Parameters (inout)	None
Parameters (out)	None
Return value	uart_status_t — always returns UART_OK.
Description	Busy-waits until TXE (SR bit 7) is set, then writes data & 0xFF to USART2->DR.
Constraints	Blocks indefinitely if the USART peripheral is not enabled or if the transmitter stalls. No timeout mechanism is implemented.
Available via	uart.h

serial_init

Function Name	serial_init
Syntax	serial_status_t serial_init(void)
Parameters (in)	None

Parameters (inout)	None
Parameters (out)	None
Return value	serial_status_t — SERIAL_OK on success.
Description	Calls uart_init() to configure USART2 at 115200 baud. No additional configuration is needed for the serial layer itself.
Constraints	None.
Available via	serial.h

serial_printf

Function Name	serial_printf
Syntax	serial_status_t serial_printf(const char *format, ...)
Parameters (in)	format — const char*: printf-style format string. Supported specifiers: %d (signed decimal), %u (unsigned decimal), %x (hexadecimal), %s (string), %c (character), %% (literal percent).
Parameters (inout)	None
Parameters (out)	None
Return value	serial_status_t — SERIAL_OK on success, SERIAL_INVALID if format is NULL.
Description	Formats the string into an internal 128-byte buffer using va_list and utils_itoa() for numeric conversions. Sends each byte with uart_write(). Null-terminates the buffer before transmitting.
Constraints	Output is silently truncated at 127 characters. Does not support field width, precision, or padding specifiers.
Available via	serial.h

4.4 Internal Function Definitions

The UART driver has no private helper functions. The Serial module also has no separate internal functions; the formatting logic is inline within serial_printf(). The utils module provides two private helpers (utils_printString and utils_printInt, both static) that are not documented here as they belong to the Utils module SDD.

5. Future Enhancements

- Receive support: add uart_read() and configure PA3 as USART2_RX (AF7) to enable bidirectional communication. This would allow the board to receive commands from the PC terminal.
- Interrupt-driven transmit: replace the polling loop in uart_write() with a circular TX buffer and the USART2 TXE interrupt, freeing the CPU for other work between byte transmissions.
- DMA transmit: connect USART2 TX to DMA1 Stream 6 Channel 4 to send entire buffers without CPU involvement, useful for high-throughput logging.
- Configurable baud rate: add a uart_setBaudRate(uint32_t baud) function that recalculates BRR at runtime, removing the compile-time dependency on a fixed 16 MHz clock.
- Multiple USART instances: extend the driver with an instance map table (similar to the ADC driver) to support USART1, USART6, or UART4/5 on the same board.
- serial_printf field width: add padding and minimum field width support (e.g., %5d) to allow aligned tabular output in the terminal monitor.

6. References

[1] Gbati, O. (2023). Embedded Systems Programming with the STM32 Microcontroller. Independently published.

[2] Mazidi, M., Chen, S., & Naimi, S. (2020). The STM32F0 ARM Microcontroller and Embedded Systems: Using Assembly and C. Microdigitaled.

[3] Deitel, P., & Deitel, H. (2016). C How to Program (8th ed.). Pearson.

[4] STMicroelectronics. (2016). RM0383: STM32F411 reference manual (Rev 3). https://www.st.com/resource/en/reference_manual/rm0383-stm32f411xce-advanced-armbased-32bit-mcus-stmicroelectronics.pdf